

Truss Analysis and Optimization

Joshua Halpert, Nicolas Weinberg

Binghamton University

ME-212 — November 23, 2025

Abstract

This project analyzes a static truss using the method of joints to determine internal member forces and reaction forces. A truss was modeled in MATLAB by defining node coordinates, member connections, constraints, and loads. The code was then adapted to solve a Pratt steel truss with three 10-kip loads, where truss height was based on the angle θ , and member cross-sectional areas were varied to find where the stress is equal to 20 ksi while minimizing overall weight, using a steel specific weight of 0.284 lb/in³. Trusses were tested for θ values ranging from 20° to 80° to identify the most efficient configuration. The study demonstrates how structural geometry influences weight and performance.

Introduction

Truss structures are used everywhere, like in bridges, roofs, cranes, and other load-bearing applications, due to their high strength and efficiency of materials. In engineering design, trusses are commonly analyzed under static loading to find where members carry tension or compression and whether the structure is strong enough to support the load. The method of joints provides a way to determine the internal forces of each member by finding the equilibrium equations at each node. While this approach is usually hand calculated for small structures, it can be calculated computationally to analyze larger trusses and evaluate multiple design factors in the project.

In this project, the method of joints was coded in MATLAB to automate equilibrium calculations using given node coordinates, member connections, constraints, and loads. It was then applied to a Pratt steel truss with three 10-kip loads, where cross-sectional areas of the members were found to support a stress of 20 ksi. Truss structures were created for different heights using angles ranging from 20° to 80°. Structural weight was then computed using the given specific weight of steel. By comparing these configurations, the study finds how geometric changes influence the total material cost and performance, leading to a more efficient structural design.

Problem Definition

A 40-ft Pratt truss must support three 10-kip loads spaced evenly. Member cross-sections are selected so that the stress does not exceed 20 ksi. The goal is to minimize structural weight.

Parameters:

- Three Downward Loads: 10 kip
- Steel density: 0.284 lb/in^3
- Allowable stress: 20 ksi
- Span: 40 ft
- Variable: Truss angle $\theta \in [20^\circ, 80^\circ]$

Technical Approach

The truss is analyzed using the method of joints, which relies on the equilibrium of forces at each node in both the x and y directions. A global system of equations is formed:

$$\mathbf{M} \cdot \mathbf{F} = \mathbf{E}$$

Here:

- $\mathbf{M}$ is the equilibrium coefficient matrix based on geometry.
- $\mathbf{F}$ is the vector of unknown member and reaction forces.
- $\mathbf{E}$ is the external force vector (loads and supports).

The MATLAB code constructs this matrix by reading data from ‘data.txt’, which defines node coordinates, connectivity, boundary conditions, and external loads. The top nodes are positioned using trigonometry based on angle θ :

$$\text{height} = 10 \cdot \frac{\tan(\theta)}{\tan(45^\circ)}$$

Solving the system gives internal forces in each member. Then, required areas are computed by rearranging the stress formula:

$$A = \frac{|F|}{\sigma_{\text{allow}}}$$

Each member's weight is:

$$w_i = A_i \cdot L_i \cdot \gamma$$

Where γ is the steel density and L_i is member length. Total weight is the sum of all member weights.

Implementation

Two MATLAB files were written:

truss.m:

- Reads input file with node coordinates, element connections, reactions, and external loads
- Builds the global equilibrium matrix and external force vector
- Solves for unknown member and reaction forces
- Prints element and reaction results

trussRun.m:

- Loops over θ values from 20° to 80°
- Recomputes node coordinates and writes to 'data.txt'
- Calls 'truss.m' to compute forces
- Computes required area per member using $\sigma = F/A$
- Uses member lengths and steel density to compute total weight
- Identifies and stores the configuration with minimum weight

Numerical Examples

At angle $\theta = 30^\circ$, the following member results were obtained:

Element	Length (ft)	Force (lb)	Area (in ²)
1-4	10.00	25980.8	1.2990
5, 8	11.55	-30000.0	1.5000
6, 7	10.00	-34641.0	1.7321
9,13	5.77	10000.0	0.5000
10,12	11.55	10000.0	0.5000
11	5.77	0.0	0.0000

Table 1: Member forces and cross-sectional areas at $\theta = 30^\circ$

Results

The minimum weight configuration occurs at $\theta = 57^\circ$:

- Total structural weight: **312.36 lb**
- All members remained within the 20 ksi stress limit

Element	Length (ft)	Force (lb)	Area (in ²)
1-4	10.00	9741.1	0.4871
5, 8	18.36	-17885.4	0.8943
6, 7	10.00	-12988.2	0.6494
9,13	15.40	10000.0	0.5000
10,12	18.36	5961.8	0.2981
11	15.40	0.0	0.0000

Table 2: Member forces and cross-sectional areas at $\theta = 57^\circ$

Angle (deg)	Weight (lb)
20	692.65
30	472.23
40	370.09
50	321.96
57	312.36
60	314.82
70	317.26
80	351.52

Table 3: Truss weight vs. angle (sample points)

Conclusion

This study showed how geometry impacts the performance and efficiency of a truss. Automating the method of joints using MATLAB enabled rapid analysis of many configurations. The minimum weight configuration was found at $\theta = 57^\circ$ with a weight of 312.36 lb. The method allows engineers to optimize designs early in development, saving time and material costs.

References

- MATLAB Documentation: <https://www.mathworks.com/help/matlab/>
- StaticsTrussProblem.PDF
- bingProject.PDF

Appendix A: truss.m

```
1 % truss - this program uses a matrix method to solve for the forces
   in
2 % the members of a statically determinate truss. It also
3 % computes the reaction forces.
4 %
5 % The program is started by typing
6 %
7 % truss('input_data_file.txt')
8 %
9 % at the MATLAB prompt. In this case the input data file is
10 % named input_data_file.txt Any file name can be used. The
11 % file is a common text file and can be created with NOTEPAD
12 %
13 % Written by: Robert Greenlee (modified/fixed version)
14
15 function A = truss(file_name)
16
17 % OPEN DATA FILE
18 fid = fopen(file_name, 'r');
19
20 % READ DATA - READ NODE DATA - First read the number of nodes in the
   truss
21 Number_nodes = fscanf(fid, '%d', 1);
22
23 % Read the coordinates for each node
24 for i = 1:Number_nodes
25     Node = fscanf(fid, '%d', 1);
26     Coordinate(Node, 1) = fscanf(fid, '%g', 1);
27     Coordinate(Node, 2) = fscanf(fid, '%g', 1);
28 end
29
30 % READ DATA - READ ELEMENT DATA - Now read the number of elements
   and the element definitions
31 Number_elements = fscanf(fid, '%d', 1);
32
33 % Save element connections to memory
34 Element_data = zeros(Number_elements, 3);
35 for i = 1:Number_elements
36     Element_data(i, :) = fscanf(fid, '%d %d %d', 3);
```

```

37 end
38
39 % READ DATA - READ REACTION DATA - Now read the number of reactions
40 Number_reactions = fscanf(fid, '%d', 1);
41
42 % Check if the problem is statically determinate
43 if (2 * Number_nodes ~= (Number_elements + Number_reactions))
44     error('Invalid number of nodes, elements, and reactions');
45 end
46
47 % Initialize matrix
48 M = zeros(2 * Number_nodes, Number_elements + Number_reactions);
49
50 % Fill in force matrix using saved element data
51 for i = 1:Number_elements
52     Element = Element_data(i, 1);
53     Node_from = Element_data(i, 2);
54     Node_to = Element_data(i, 3);
55
56     dx = Coordinate(Node_to, 1) - Coordinate(Node_from, 1);
57     dy = Coordinate(Node_to, 2) - Coordinate(Node_from, 2);
58     Length = sqrt(dx^2 + dy^2);
59
60     M(2 * Node_from - 1, Element) = dx / Length;
61     M(2 * Node_to - 1, Element) = -dx / Length;
62     M(2 * Node_from, Element) = dy / Length;
63     M(2 * Node_to, Element) = -dy / Length;
64 end
65
66 % READ DATA - Add the reactions to the matrix
67 for i = 1:Number_reactions
68     Reaction = fscanf(fid, '%d', 1);
69     Node = fscanf(fid, '%d', 1);
70     Direction = fscanf(fid, '%s', 1);
71
72     col = Number_elements + Reaction;
73
74     if strcmpi(Direction, 'Y')
75         M(2 * Node, col) = M(2 * Node, col) + 1;
76     elseif strcmpi(Direction, 'X')
77         M(2 * Node - 1, col) = M(2 * Node - 1, col) + 1;

```

```

78     else
79         error('Invalid direction for reaction');
80     end
81 end
82
83 % READ DATA - READ EXTERNAL FORCE DATA - Now read in the external
    forces
84 External = zeros(2 * Number_nodes, 1);
85 Number_forces = fscanf(fid, '%d', 1);
86 for i = 1:Number_forces
87     Node = fscanf(fid, '%d', 1);
88     Force = fscanf(fid, '%g', 1);
89     Direction = fscanf(fid, '%g', 1);
90     External(2 * Node - 1) = External(2 * Node - 1) - Force * cosd(
        Direction);
91     External(2 * Node)      = External(2 * Node)      - Force * sind(
        Direction);
92 end
93
94 % Close file
95 fclose(fid);
96
97 % COMPUTE FORCES - Solve the system of equations
98 A = M \ External;
99
100 % REPORT FORCES - FORCES IN THE ELEMENTS
101 for i = 1:Number_elements
102     fprintf('Element %d = %g \n', i, A(i))
103 end
104
105 % REPORT FORCES - REACTION FORCES
106 for i = 1:Number_reactions
107     fprintf('Reaction %d = %g \n', i, A(Number_elements + i))
108 end
109
110 end % End of program

```


Appendix B: trussRun.m

```
1 function trussRun()
2 clc;
3 clear;
4 close all;
5
6 thetas = 20:1:80;      % test angles
7 stress_allow = 20000;  % psi
8 density = 0.284;      % lb/in^3
9 scale = 1000;          % convert 10 lb loads      10,000 lb
10 basefile = 'data.txt';
11 weights = zeros(length(thetas),1);
12 % loop for each theta value
13 for t = 1:length(thetas)
14     theta = thetas(t);
15     fprintf("\n %d deg \n",theta);
16     % read file
17     fid = fopen(basefile,'r');
18     nn = fscanf(fid,'%d',1);
19     coords = zeros(nn,2);
20     for i = 1:nn
21         node = fscanf(fid,'%d',1);
22         coords(node,1) = fscanf(fid,'%g',1);
23         coords(node,2) = fscanf(fid,'%g',1);
24     end
25     ne = fscanf(fid,'%d',1);
26     elems = zeros(ne,2);
27     for i = 1:ne
28         fscanf(fid,'%d',1);
29         elems(i,1) = fscanf(fid,'%d',1);
30         elems(i,2) = fscanf(fid,'%d',1);
31     end
32     % save the rest
33     rest = {};
34     while ~feof(fid)
35         line = fgetl(fid);
36         rest{end+1} = line;
37     end
38     fclose(fid);
39     % fix truss scale
```

```

40 span_units = coords(5,1) - coords(1,1);
41 ft_per_unit = 40 / span_units;
42 % move the top nodes based on theta
43 height_ft = 10 * tand(theta) / tand(45);
44 height_units = height_ft / ft_per_unit;
45
46 for i = 1:nn
47     if coords(i,2) > 0
48         coords(i,2) = height_units;
49     else
50         coords(i,2) = 0;
51     end
52 end
53 % make a temp file
54 fid = fopen('temp.txt','w');
55 fprintf(fid,"%d\n",nn);
56 for i = 1:nn
57     fprintf(fid,"%d %.6f %.6f\n",i,coords(i,1),coords(i,2));
58 end
59
60 fprintf(fid,"%d\n",ne);
61 for i = 1:ne
62     fprintf(fid,"%d %d %d\n",i,elems(i,1),elems(i,2));
63 end
64
65 for i = 1:length(rest)
66     fprintf(fid,"%s\n",rest{i});
67 end
68 fclose(fid);
69 % run the truss program
70 raw = truss("temp.txt");
71 forces = raw(1:ne) * scale; % now lb
72 % find lengths
73 len_ft = zeros(ne,1);
74 for i = 1:ne
75     n1 = elems(i,1); n2 = elems(i,2);
76     dx = (coords(n2,1)-coords(n1,1)) * ft_per_unit;
77     dy = (coords(n2,2)-coords(n1,2)) * ft_per_unit;
78     len_ft(i) = sqrt(dx^2 + dy^2);
79 end
80 len_in = len_ft * 12;

```

```

81     % find areas
82     areas = abs(forces) ./ stress_allow;
83     % find weights
84     volumes = areas .* len_in;
85     weights(t) = (sum(volumes))*density;
86     % print the results
87     fprintf("Member Results:\n");
88     for i = 1:ne
89         fprintf("Elem %2d: L = %6.2fft Force = %10.1f lb Area =
90         %.4f in^2\n", i, len_ft(i), forces(i), areas(i));
91     end
92     fprintf("Weight = %.2f lb\n", weights(t));
93 end
94 % find optimal
95 [minW, idx] = min(weights);
96 fprintf("\n OPTIMAL \n");
97 fprintf("      = %d\n", thetas(idx));
98 fprintf("Weight = %.2f lb\n\n", minW);
99 end

```

Appendix C: data.txt

```
1 8
2 1    0  0
3 2    10 0
4 3    20 0
5 4    30 0
6 5    40 0
7 6    10 10
8 7    20 10
9 8    30 10
10 13
11 1    1  2
12 2    2  3
13 3    3  4
14 4    4  5
15 5    1  6
16 6    6  7
17 7    7  8
18 8    8  5
19 9    2  6
20 10   6  3
21 11   3  7
22 12   3  8
23 13   4  8
24 3
25 1    1  X
26 2    1  Y
27 3    5  Y
28 3
29 2    10 270
30 3    10 270
31 4    10 270
```